

7 - Eventos

¿Qué es un evento?

Un **evento** es cualquier acción o suceso que ocurre dentro del sistema y que el navegador nos suministra para que podamos usarlos:

- El usuario hace clic con el mouse sobre un elemento determinado o coloca el cursor sobre un elemento determinado.
- El usuario presiona una tecla en el teclado.
- El usuario cambia el tamaño o cierra la ventana del navegador.
- Una página web termina de cargar.
- Un formulario se envía
- Un video se reproduce, pausa o finaliza la reproducción.
- Un error ocurre.
- etc. [Hay muchísimos eventos más](#) que puedes consultar.

Por ejemplo, cuando un usuario hace clic en un botón, podrías querer cambiar el color de fondo de la página. Este clic es el **evento** que desencadena una acción.

```
1<button id="changeColor">Cambiar color</button>
2<script>
3  document.getElementById("changeColor").addEventListener("click", () => {
4    document.body.style.backgroundColor = "lightblue";
5  });
6</script>
```

Nota:

Los eventos en el navegador no son parte del núcleo de JavaScript, sino que forman parte de las API del navegador.

Objeto Event

Como decíamos antes, este tipo de acciones hacen que el navegador nos informe de ellas a través de un objeto conocido como **El Objeto Event**. Este objeto contiene toda la información acerca del evento que acaba de producirse, tiempo, contenido, tipo, métodos de control, listeners asociados, etc. Aquí podemos ver algo de la información que contiene:

Tipo	Nombre	Descripción
Propiedades de control del evento	type	Devuelve el tipo de evento producido, sin el prefijo on (p.ej. click)
target	Devuelve el elemento del DOM que disparó el evento (inicialmente)	
currentTarget	Devuelve el elemento del DOM que está disparando el evento actualmente (no necesariamente el elemento que disparó el evento, ya que puede ser un disparo debido a burbujeo)	
Otras propiedades de control del evento	eventPhase (indica en qué fase de tratamiento de evento estamos, 1 captura, 2 en objetivo, 3 burbujeo), bubbles (booleana, indica si es un evento que burbujea o no), cancelable (booleana, devuelve si el evento viene seguido de una acción predeterminada que puede ser cancelada), cancelBubble (booleana, devuelve si el evento actual se propagará hacia arriba en la jerarquía del DOM o no).	
Propiedad temporal	timeStamp	Devuelve una medida de tiempo en milisegundos desde un origen temporal determinado.
Propiedades de localización del puntero del ratón	clientX, clientY	Devuelven las coordenadas en que se encontraba el puntero del ratón cuando se disparó el evento. Las coordenadas están referidas a la esquina superior izquierda de la ventana del navegador y se expresan en píxeles.
screenX, screenY	Devuelven las coordenadas en que se encontraba el puntero del ratón cuando se disparó el evento. Las coordenadas están	

	referidas a la esquina superior izquierda de la pantalla y se expresan en pixeles.	
pageX, pageY	Devuelven las coordenadas en que se encontraba el puntero del ratón cuando se disparó el evento. Las coordenadas están referidas a la esquina superior izquierda del documento, que pueden ser distintas a las de la ventana si el usuario ha hecho scroll sobre el documento.	
Propiedad para detectar el botón del ratón pulsado	button	Normalmente empleado para el evento mouseup (liberación de botón del ratón) para detectar cuál ha sido el botón pulsado. Contiene un valor numérico: 0 para click normal (botón izquierdo), 1 para botón central (botón en el scroll), 2 para botón auxiliar (botón derecho).
Propiedades relacionadas con el teclado	Para determinar qué tecla ha sido pulsada	Lo estudiaremos por separado en la siguiente entrega del curso
Propiedades relacionadas con drag and drop	Algunas no estandarizadas	dataTransfer, dropEffect, effectAllowed, files, types
Otras propiedades varias	Otras (algunas no estandarizadas)	x, y, layerx, layery, offsetX, offsetY, wheelDelta, detail, relatedNode, relatedTarget, view, attrChange, attrName, newValue, prevValue, data, lastEventId, origin, source
Método	stopPropagation()	Detiene la propagación del evento
Método	preventDefault()	Cancela (si es posible) la acción de defecto que debería ocurrir después del evento (equivale a return false para cancelar la acción).
Otros métodos	initEvent(a, b, c) se usa para definir eventos. No lo estudiaremos.	

Listener

Los listeners son “alarmas” que utilizamos para que en caso de producirse un evento concreto, se desencadene una acción que nosotros hayamos definido.

Existen muchos tipos de listeners diferentes y dos formas principales de asignarlos en JavaScript: mediante argumentos en la etiqueta del documento HTML (como `onclick`) o utilizando el método `addEventListener`. Aunque ambos enfoques logran un resultado similar, tienen diferencias clave:

- El uso de argumentos es más rápido y sencillo en el código, pero sólo nos permite poner un handler por evento, porque cualquier asignación posterior sobrescribirá la anterior.
- El uso del método nos permite poner múltiples handlers, controlar el bubbling, el capturing, etc.

Vamos a probar con un listener por argumento de tipo `onclick` en un botón. Este llamará a una función para controlar el evento (**Handler**) definida por nosotros `nextPage()`

```
<button id="boton" onclick="nextPage()">+</button>
```

```
let page = 1

function nextPage(){
  page++
}
```

Esta función ejecutará un código concreto que nosotros habremos definido y que se ejecutará, por tanto, a demanda del listener, es decir, cuando ocurra el evento. (El usuario pincha en el botón y ejecuta el código).

También podemos agregar nosotros un listener a un elemento ya creado en nuestro DOM con el método `addEventListener()`:

```
.addEventListener('listener', callback)
```

Usando el ejemplo de antes haríamos:

```
document.getElementById("boton").addEventListener('click', ()=>{page++})
```

De esta forma habremos acoplado el listener a un evento "click" en el nodo cuyo id es "boton" en nuestro DOM y le hemos indicado, mediante una función flecha, que cuando se dispare el mismo, nos aumente el valor de la variable "page".

Propagación de eventos (Event bubbling)

Cuando ocurre un evento, este viaja a través del DOM siguiendo estas fases:

1. **Captura:** El evento se propaga desde el elemento raíz hacia el objetivo.
2. **Objetivo:** El evento alcanza el elemento objetivo.
3. **Burbujeo:** El evento viaja de regreso hacia el elemento raíz.

El "burbujeo" es la propagación que se produce cuando un evento en un elemento hijo es escuchado por un listener que está puesto en un elemento padre ya que al contener uno al otro, cuando se dispara el evento en el hijo en realidad, también está disparándose en el padre que lo contiene.

Para prevenir el fenómeno del bubbling, utilizamos uno de los métodos que contiene el objeto Event natural del propio evento, el `.stopPropagation()`:

Ejemplo de burbujeo:

```
const parent = document.getElementById("parent");
const child = document.getElementById("child");

parent.addEventListener("click", () => {
  alert("Hiciste clic en el elemento padre");
});

child.addEventListener("click", (e) => {
  alert("Hiciste clic en el elemento hijo");
  e.stopPropagation(); // Detiene el burbujeo
});
```

En este caso, el clic en el elemento hijo no se propagará al padre debido al uso de `.stopPropagation()`.

Por contra, si nos ocurre de arriba hacia abajo estaríamos hablando de "capturing" aunque esto no ocurre naturalmente, sino que hay que indicarle al listener que queremos capturar los eventos de los elementos del DOM descendientes, pasándole como tercer argumento al método, un objeto de configuración:

```
document.getElementById('mi_div').addEventListener('click', ()=>console.log('div clickado!!!'), { capture: true })
```

Eliminar un listener:

Al igual que podemos añadir un listener a un elemento de nuestro dom para capturar sus eventos, podemos, llegado el momento querer quitarlo porque ya no vamos a necesitarlo. Al ser un proceso que queda pendiente, es buena práctica ir eliminándolos a medida que no vayan siendo necesarios para liberar capacidad de proceso:

```
function handleClick() {
  alert("Este controlador será eliminado");
  button.removeEventListener("click", handleClick);
}

button.addEventListener("click", handleClick);
```

- Usa `addEventListener` en lugar de atributos HTML para manejar eventos.
- Evita mezclar JavaScript con HTML.
- Usa delegación de eventos cuando trabajes con muchos elementos.
- Remueve los manejadores no utilizados para liberar recursos.